JavaScript Async Debugging

Back to JS page

Table of Contents

Log Events

Use `console.log()` to track the flow of async operations. This helps you understand the order of execution:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Log Events</h4>
<p id="demo"></p>

<script>
console.log("1. Script starts");

async function fetchData() {
  console.log("2. Inside async function");
  let result = await new Promise(function(resolve) {
    setTimeout(function() {
      console.log("3. Promise resolved");
      resolve("Data loaded");
    }, 1000);
  });
  console.log("4. After await");
  document.getElementById("demo").innerHTML = result;
}

fetchData();
console.log("5. After calling fetchData (not blocked)");
</script>

</body>
</html>
```

Example 1

Result View Example

Rule 1: Always Handle Errors

Always use `try/catch` around async operations to catch errors:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Rule 1: Always Handle Errors</h4>
<p id="demo"></p>

<script>
async function fetchWithCatch() {
  try {
    let response = await fetch('https://jsonplaceholder.typicode.com/users/1');
    if (!response.ok) {
      throw new Error("HTTP " + response.status);
    }
    let data = await response.json();
    document.getElementById("demo").innerHTML =
      "Success: " + data.name + "<br>" +
      "Email: " + data.email;
  } catch(error) {
    document.getElementById("demo").innerHTML =
      "Error caught: " + error.message + "<br>" +
      "Tip: Check the URL or network connection";
  }
}

fetchWithCatch();
</script>

</body>
</html>
```

Example 2

Result View Example

Rule 2: Check response.ok

`fetch()` does NOT reject on HTTP errors (like 404). Always check `response.ok`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Rule 2: Check response.ok</h4>
<p id="demo"></p>

<script>
async function checkResponse() {
  // Try with an invalid URL
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');

  console.log("Status:", response.status);
  console.log("ok:", response.ok);

  if (!response.ok) {
    document.getElementById("demo").innerHTML =
      "response.ok is " + response.ok + "<br>" +
      "Status: " + response.status + " " + response.statusText + "<br>" +
      "Always check response.ok after fetch!";
    return;
  }

  const data = await response.json();
  document.getElementById("demo").innerHTML = "Name: " + data.name;
}

checkResponse();
</script>

</body>
</html>
```

Example 3

Result View Example

Rule 3: Log the Right Things

Log meaningful information to understand what's happening:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Rule 3: Log the Right Things</h4>
<p id="demo"></p>

<script>
async function logCorrectly() {
  const url = 'https://jsonplaceholder.typicode.com/users/1';

  // Log the URL being fetched
  console.log("Fetching:", url);

  try {
    const response = await fetch(url);

    // Log response status
    console.log("Response status:", response.status);

    if (!response.ok) {
      throw new Error("HTTP error " + response.status);
    }

    const data = await response.json();

    // Log the received data structure
    console.log("Data received:", data);
    console.log("User name:", data.name);

    document.getElementById("demo").innerHTML =
      "Check the browser console (F12) for logs<br>" +
      "User: " + data.name + "<br>" +
      "Email: " + data.email;

  } catch(error) {
    console.error("Error:", error.message);
    document.getElementById("demo").innerHTML = "Error: " + error.message;
  }
}

logCorrectly();
</script>

</body>
</html>
```

Example 4

Result View Example

Understand Execution Order

Async code does not execute in the order it appears. Use logs to understand the flow:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Debugging</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Understand Execution Order</h4>
<p id="demo"></p>

<script>
console.log("A: Start of script");

setTimeout(function() {
  console.log("B: setTimeout callback");
}, 0);

Promise.resolve().then(function() {
  console.log("C: Promise microtask");
});

async function test() {
  console.log("D: Inside async function (before await)");
  await Promise.resolve();
  console.log("E: After await");
}
test();

console.log("F: End of script (sync code)");

// Expected console order: A, D, F, C, E, B
document.getElementById("demo").innerHTML =
  "Check browser console (F12) for execution order:<br>" +
  "Expected: A → D → F → C → E → B<br><br>" +
  "A: Start<br>" +
  "D: Inside async (before await)<br>" +
  "F: End of sync code<br>" +
  "C: Promise microtask<br>" +
  "E: After await<br>" +
  "B: setTimeout (Task queue - last)";
</script>

</body>
</html>
```

Example 5

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Async Debugging